

RO 3-292 29.10.8.3 [simd.comparison] Make basic_simd a Regular Type (with Boolean operator==)

Document number: P3925R0

Date: 2025-11-07

Authors: Darius Neațu <dariusn@adobe.com>, Andrei Alexandrescu <andrei@nvidia.com>, Lucian Radu Teodorescu <lucteo@lucteo.ro>, Radu Nichita <radunichita99@gmail.com>

Audience: Evolution

Tagline: Define operators primarily to mimic conventional usage.

Abstract

The current design of `std::simd` equality semantics, introduced by P1928 [1], diverges from established Regular type conventions in C++. The inconsistency undermines predictability and interoperability with generic code.

This paper motivates the reconsideration of `std::simd` equality semantics to restore consistency with Regular type behavior across the C++ Standard Library.

It examines three alternative design directions in response to National Body comment RO 3-292.

Relation to Prior Work

This document is intended as a companion and follow-up to [P2892R0] ("`std::simd` Types Should Be Regular" - [2]), providing additional motivation, user-centered analysis, and design-space exploration.

While P2892R0 establishes the technical case for restoring Regular semantics to `std::simd`, the present paper expands the discussion along usability, pedagogy, and design-space dimensions.

The goal is not to restate P2892R0, but to complement it by strengthening the rationale for a design direction that is technically sound, pedagogically coherent, and aligned with established C++ design philosophy.

Motivation

Principle: Make concrete types regular.

Although `std::simd` is a new addition to the C++ Standard, the STL already contains several types that represent sequences of values - such as `std::bitset`, `std::array`, `std::vector`, and `std::list`. All of them meet regularity, except `std::valarray`, which is rarely used.

All of these follow Regular semantics, providing equality operators that return `bool` and integrate seamlessly with generic algorithms.

When we hear “SIMD is weird,” the natural implication is not that it deserves a bespoke set of operator semantics that contradict established C++ standards and intuition, but rather that it is a perfect candidate for functions and named APIs where the uncommon behavior is explicit (`equal_elements` , mask operations, etc.). Overloading `operator==` with semantics that differ from the rest of the language is not a local, isolated exception; it is a precedent, and precedents are how accidental design patterns are born and must be approached with trepidation. (Just consider `std::vallarray` .) If we bless `std::simd` as “the one where `==` doesn’t behave like `==` ,” we implicitly legitimize similar departures elsewhere, weakening one of C++’s few remaining points of semantic stability. That kind of shift should be approached with trepidation, not treated as an implementation detail for a niche facility: the burden of proof lies with any proposal that teaches users that a familiar operator suddenly means something different.

New Information in LEWG

Since the publication of P2892R0 (“`std::simd` Types Should Be Regular”), new evidence, implementation experience, and educational feedback have emerged that justify reopening discussion of `std::simd` equality semantics.

The following points summarize the additions and updates introduced by this paper.

New Empirical and User Evidence

Feedback gathered from implementers, educators, and industrial adopters of experimental SIMD libraries (e.g., in graphics and embedded domains) demonstrates that the current non-Regular equality model (`operator==` returning a mask) is unintuitive for most C++ developers.

Multiple real-world cases show compile-time surprises in generic algorithms (`std::find` , `std::erase` , `std::ranges::equal`) and in codebases using defaulted equality.

Clarified User Impact and Teachability Analysis

This paper distinguishes between SIMD experts (for whom element-wise comparison is natural) and mainstream C++ developers, who expect `==` to yield a scalar `bool` .

This sociotechnical clarification was absent from P2892R0 and reframes the issue as one of teachability and user trust, not only language purity.

We conducted an internal survey at the University POLITEHNICA of Bucharest among faculty members involved in C++ teaching, and there was strong consensus that the current design of `std::simd` equality semantics breaks the established Regularity and usability principles discussed in this paper.

We also included non-C++ instructors and software engineering educators in the survey to assess general comprehensibility. The broader feedback reinforced the same conclusion: the current `std::simd` equality semantics are difficult to explain and violate expected consistency with other value-semantic types.

Romanian NB strongly advises to take into account C++ users who are not SIMD experts, who represent a larger group than SIMD specialists.

Expanded Design Space

The paper introduces two concrete design alternatives (Regular equality, explicit-only element-wise comparison) and one completeness-only fallback (removal or deferral of `std::simd`).

These structured options offer the committee a clear basis for decision-making beyond the single proposal explored in P2892R0.

Romanian NB strongly recommends taking into account the full design space. We believe the current design can be improved to better serve a broader set of users.

Integration with Current Standardization Context

Since P2892R0, the Parallelism TS types have been integrated into the C++ Working Draft, and `std::simd` now participates directly in Library Evolution discussions.

Proposed Solutions

All three options are analyzed from the perspectives of usability, teachability, standard consistency, and library design coherence, with the recommendation that either Version 1 or Version 2 be adopted to preserve clarity and Regularity in modern C++.

v1: Rename Existing API (least invasive change)

Version 1 proposes the simplest and least invasive change - removing or renaming `operator==`, so that element-wise comparison is performed only through the existing explicit free function `equal_elements`. This avoids ambiguity and requires no API additions.

v2: Renaming + Restoring `std::simd` Regularity

Version 2 proposes restoring Regular behavior by ensuring `operator==` returns `bool`, thus enabling predictable use in generic algorithms and supporting defaulted equality for user-defined types.

v3: Move `std::simd` to Another Shipping Vehicle

Version 3, included only for completeness, considers the removal or deferral of `std::simd` from the working draft if consensus on equality semantics cannot be reached.

Possible future shipping vehicles:

- C++29
- A standalone White Paper
- A standalone Technical Specification (i.e., move `std::simd` back into a TS).

Wording

The specific edits to `[simd.comparison]` in the Working Draft are trivial and intentionally omitted in this revision.

Both Version 1 and Version 2 require only minimal, mechanical changes to the wording.

No normative behavior outside `[simd.comparison]` is affected.

References

- [1] `std::simd` — merge data-parallel types from the Parallelism TS 2. Matthias Kretz. <http://wg21.link/P1928> (<http://wg21.link/P1928>)
- [2] `std::simd` Types Should Be Regular. Joe Jevnik, David Sankel. <http://wg21.link/P2892R0> (<http://wg21.link/P2892R0>)
- [3] C++ Core Guidelines. Bjarne Stroustrup, Herb Sutter. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>)